

Python & Tools Cheat Sheet

Week 5: Terminal, Git, and File I/O Mastery

1. The Terminal (Command Line)

COMMAND	ACTION	EXAMPLE
<code>ls</code>	List all files in current folder	<code>ls</code>
<code>cd</code>	Change directory (move to folder)	<code>cd projects</code>
<code>mkdir</code>	Make a new directory (folder)	<code>mkdir week5_lab</code>
<code>touch</code>	Create a new empty file	<code>touch main.py</code>
<code>rm</code>	Remove/Delete a file	<code>rm old_code.py</code>
<code>python</code>	Run a python script	<code>python main.py</code>

Tip: Use the `Tab` key to auto-complete file and folder names!

2. Git & GitHub Workflow

COMMAND	WHAT IT DOES
<code>git init</code>	Turn the current folder into a Git repository.
<code>git add .</code>	Stage all changes (prepare them to be saved).
<code>git commit -m "..."</code>	Save your changes locally with a descriptive message.
<code>git push</code>	Upload your local saves to your GitHub account.
<code>git status</code>	Check which files are changed or staged.

3. Python File I/O (Read & Write)

Always use the `with` keyword to ensure files close properly.

MODE	CODE SNIPPET
Write	<pre>with open("file.txt", "w") as f: f.write("Hello!")</pre>
Read	<pre>with open("file.txt", "r") as f: content = f.read()</pre>

Append

```
with open("file.txt", "a") as f:  
    f.write("\nNew Line")
```

4. Modularity & Imports

To use functions from `tools.py` inside `main.py`:

```
# In main.py  
import tools  
tools.my_function()
```

To turn a folder into a **Package**, add a blank file named `__init__.py` inside it.